

Assignmen4_Part1_KoorosMoabber_Ver8

April 17, 2026

1 Automated Track Infrastructure Recognition by Using Vibration Analysis

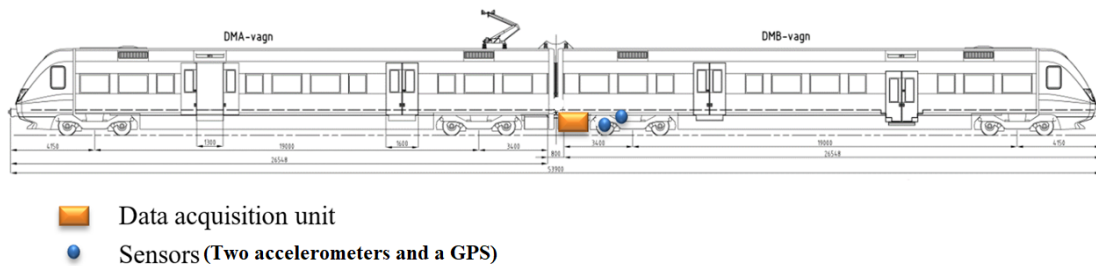
Kooros Moabber Luleå University, Luleå, Sweden and Volvo Cars Corporation, Gothenburg, Sweden

1.1 Introduction

Track infrastructure monitoring is critical for optimizing KPIs such as efficiency, cost, availability and safety. This monitoring is more crucial when the infrastructure is in a harsh environment and wore out due to age. Traditional maintenance is based on periodic inspections which is time consuming and costly and should be replaced with a predictive maintenance where an AI-driven approach for automated real time track monitoring is one of the best approaches. This monitoring can be done on all elements of infrastructure particularly turnouts, joins and bridges that are more vulnerable for any problem.

An approach for this monitoring is to measure the vibration detected caused by vehicle on track by accelerometer in a specific infrastructure and combine this vibration data with GPS and expected/predefined infrastructure information to detect infrastructure elements (events) and compare their acceleration pattern with a normal pattern to detect any abnormally.

In this report, we have shown the mentioned elements on a standard map to be used for future investigation.



Figure

1: Measurement System Implementation

1.2 Method

The methodology begins with installing the necessary tools such as accelerometers and GPS on a train to measure data.

In the first section, measured data files is used to load infrastructure information (including turnouts, joints, and bridges) and plot them on the map. This map is saved as an static image and

will be discussed.

In the second part, the spatial position to time-synchronous vibration data is linked interactively to detect/extract any event that shows elements in the selected infrastructure.

In the third part each element is labeled as Bridge, Rail-joint, Turnout and other and data is saved in the final data frame. The final dataset has been manipulated to remove unnecessary data (filtering), normalize data and finally visualized.

1.3 Results and Analysis

For extracting data and analyzing them, first data files was read to extract map with all elements.

Let's start by importing the necessary packages (mostly from assignment 3):

```
[1]: import numpy as np
import pandas as pd
import plotly.graph_objects as go
import warnings
import kaleido
from IPython.display import HTML, display, Markdown
import matplotlib
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import norm
from scipy.stats import skew
from scipy.stats import shapiro
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_validate
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.linear_model import Lasso
from sklearn.ensemble import RandomForestClassifier
```

1.3.1 Data Extraction and Manipulation

Data files which are included three files, each containing coordinates of that specific element. lets define the the file path in a library:

```
[2]: files = {
    "Bridge": "Data1/converted_coordinates_Resultat_Bridge.csv",
    "RailJoint": "Data1/converted_coordinates_Resultat_RailJoint.csv",
    "Turnout": "Data1/converted_coordinates_Turnout.csv"
}
```

All these three files are read in a for loop and after stripping names column, only data of latitude and longitude are read (Northing and easting are discarded) and converted to numeric and other columns are removed. A Category field is added to save the element type (Bridge, RailJoint and

Turnout). An accumulator for frames (data_frames) has been generated to append all data frame before concatenating them to make a single data frame (data).

```
[3]: data_frames = []
for category, file in files.items():
    print(category)
    print(file)
    try:
        df = pd.read_csv(file, encoding="utf-8")
        df.columns = df.columns.str.strip()
        if "Latitude" in df.columns and "Longitude" in df.columns:
            df["Latitude"] = pd.to_numeric(df["Latitude"], errors="coerce")
            df["Longitude"] = pd.to_numeric(df["Longitude"], errors="coerce")
            df = df[["Latitude", "Longitude"]]
            df["Category"] = category
            data_frames.append(df)
            print(f"Successfully loaded {category} data: {len(df)} rows")
        else:
            print(f"Warning: {category} file does not contain 'Latitude' and
↳ 'Longitude' columns.")
            print(f"Available columns: {df.columns.tolist()}")
        except Exception as e:
            print(f"Error loading {category}: {e}")
if data_frames:
    data = pd.concat(data_frames, ignore_index=True)
else:
    raise ValueError("No valid data found. Check your CSV files.")
```

Bridge

Data1/converted_coordinates_Resultat_Bridge.csv

Successfully loaded Bridge data: 25 rows

RailJoint

Data1/converted_coordinates_Resultat_RailJoint.csv

Successfully loaded RailJoint data: 20 rows

Turnout

Data1/converted_coordinates_Turnout.csv

Successfully loaded Turnout data: 75 rows

There are 25 bridges, 20 rail joints and 75 turnouts (elements) in our investigations.

Datasets can briefly be investigated by the following commands to understand the properties/attributes of data. Besides all missing latitude and longitude data records (any “NA”) are eliminated (which are zero here).

```
[4]: print("Data counts per category:\n", data["Category"].value_counts()) #
↳ Debugging: Check if all categories exist
print("Data summary:\n", data.describe()) # Check if latitude and longitude
↳ values are valid
```

```
print("Missing values per column:\n", data.isnull().sum()) # Check for missing
↳ values in the data
data = data.dropna(subset=["Latitude", "Longitude"]) # Drop rows with missing
↳ Latitude or Longitude values (if any)
```

Data counts per category:

```
Category
Turnout      75
Bridge       25
RailJoint    20
Name: count, dtype: int64
Data summary:
      Latitude  Longitude
count  120.000000  120.000000
mean    60.792431   14.921160
std      0.191516   0.284255
min     60.510878   14.518605
25%     60.587961   14.570193
50%     60.748120   15.064924
75%     61.007341   15.120486
max     61.009065   15.352741
```

Missing values per column:

```
Latitude      0
Longitude     0
Category      0
dtype: int64
```

Total elements is 120 with specific range (min and max) that can be used for filtering of data outside of the range. At the end, marker styles with different colors and sizes are defined and the first two records of each element are shown.

```
[5]: marker_styles = {
    "Bridge": {"color": "red", "size": 5},
    "RailJoint": {"color": "blue", "size": 4},
    "Turnout": {"color": "green", "size": 6}
}

for category in marker_styles.keys(): # Add additional debugging to check data
↳ before plotting
    category_data = data[data["Category"] == category]
    print(f"{category}: {len(category_data)} rows")
    if len(category_data) > 0:
        print(f"Sample coordinates for {category}:")
        print(category_data[["Latitude", "Longitude"]].head(2))
    else:
        print(f"WARNING: No data for {category}!")
```

Bridge: 25 rows

Sample coordinates for Bridge:

```

    Latitude Longitude
0  60.529364  15.295477
1  60.535942  15.280320
RailJoint: 20 rows
Sample coordinates for RailJoint:
    Latitude Longitude
25  60.54932  15.271503
26  60.55242  15.256577
Turnout: 75 rows
Sample coordinates for Turnout:
    Latitude Longitude
45  60.687000  15.112570
46  60.687337  15.111958

```

for making a map with all these elements, a figure handler is used and all these elements as a separate trace based on their altitude, longitude and category are added as a marker to it. Since Scattermapbox is an old command, a warning is shown that has been eliminated by added command.

```

[6]: warnings.filterwarnings("ignore")
fig = go.Figure()
for category, style in marker_styles.items():
    category_data = data[data["Category"] == category]

    if len(category_data) > 0:
        fig.add_trace(go.Scattermapbox(
            lat=category_data["Latitude"],
            lon=category_data["Longitude"],
            mode="markers",
            marker=dict(
                color=style["color"],
                size=style["size"]
            ),
            name=category
        ))
    else:
        print(f"Skipping {category} - no data available")

```

Finally a free map (open-street-map) is used to show all these elements which are marked previously in figure handler. The size of map, margins and zoom factor is set to show whole elements and final figure is saved as a .png static file.

```

[7]: fig.update_layout(
    mapbox_style="open-street-map",
    title="Railway Map with Bridges, Joints and Turnouts",
    legend_title_text="Legend",
    width=1200, # Set the width of the figure in pixels
    height=800, # Set the height of the figure in pixels
    margin={"r": 0, "t": 50, "l": 50, "b": 0},

```

```

mapbox=dict(
    zoom=8.5,
    center=dict(lat=data["Latitude"].mean(), lon=data["Longitude"].mean())
    # Center map around the data
)

if len(fig.data) == 0: # Check if we have any traces
    print("WARNING: No valid traces were added to the figure. Check your data!")

fig.show(renderer="iframe") # Show the figure

```

This figure illustrates various data points, including the turnout (represented by a green circle), the rail joint (depicted by a blue circle), and the bridge (indicated by a red circle), all located between Mora and Borlänge station.

```

[8]: fig.write_image(
    r"E:\assignment4\railway_map.png",
    width=1200,      # pixels
    height=800,      # pixels
    scale=2          # scale=2 doubles resolution (good for reports)
)

```

```

[9]: file_name_data="Data1/gps_marker_data.csv"
    data.to_csv(file_name_data, index=False)

```

1.4 Discussion and Summary

In this report, the elements including the turnout (represented by a green circle), the rail joint (depicted by a blue circle), and the bridge (indicated by a red circle), all located between Mora and Borlänge station are extracted and demonstrated. The concentrations of elements are different point to point that should be considered in the final report.

Railway Map with Bridges, Joints and Turnouts

